CSE344 – System Programming
Homework #1 Report

`myFind` – Advanced File Search Program

Spring 2026

Contents

1 Problem Description **2**

2 How I Solved the Problem **2**
- 2.1 Overall Architecture . . . 2
- 2.2 Data Structures . . . 2
- 2.3 Algorithm Flow . . . 3

3 Design Decisions **3**
- 3.1 Custom Regex Implementation ('+' Operator) . . . 3
- 3.2 lstat() vs stat() . . . 4
- 3.3 Linked List for Tree Output . . . 4
- 3.4 Permission Parsing . . . 4
- 3.5 Signal Safety . . . 4
- 3.6 Memory Management . . . 4
- 3.7 System Call Error Handling . . . 4

4 Requirements Achieved **5**

5 Requirements Not Achieved **5**

6 Compilation and Testing **6**
- 6.1 Compilation . . . 6
- 6.2 Test Examples . . . 6

7 File Structure **6**

Problem Description

The objective of this homework is to implement an advanced file search program called `myFind` for POSIX-compatible operating systems. The program recursively traverses a given directory tree and searches for files matching user-specified criteria. Results are displayed in a formatted tree structure. The program is written in C using only POSIX system calls and standard library functions.

How I Solved the Problem

Overall Architecture

The program follows a pipeline architecture with clearly separated responsibilities:

```
main()
  |-- Argument Parsing     (getopt)
  |-- Signal Setup          (sigaction for SIGINT)
  |-- Recursive Search      (opendir / readdir / lstat)
  |     |-- Filter Engine   (matches_criteria)
  |     |     |-- match_name()   (custom regex with '+')
  |     |     |-- match_size()   (exact byte comparison)
  |     |     |-- match_type()   (POSIX S_IS* macros)
  |     |     |-- match_perm()   (permission bitmask comparison)
  |     |     |-- match_links()  (hard link count comparison)
  |     |-- Tree Builder    (linked list with ancestor marking)
  |-- Result Printer        (formatted tree output)
  |-- Cleanup               (free all allocated memory)
```

Data Structures

Two primary structures are used:

SearchCriteria: Holds all filter values and `use_*` flags indicating which filters are active. This allows any combination of filters to be applied using AND logic.

```c
typedef struct {
    char *filename_pattern;   // -f filter value
    off_t file_size;          // -b filter value
    char file_type;           // -t filter value
    char permissions[10];     // -p filter value
    int link_count;           // -l filter value
    int use_name, use_size, use_type, use_perm, use_links;
} SearchCriteria;
```

TreeNode: A singly linked list node used to collect directory traversal results. Each node stores the entry name, its depth in the tree, and an `is_dir` flag with three states:

- `0` = matching file (always printed)

- 1 = directory (not yet marked for printing)
- 2 = directory (marked for printing — either matches criteria or is an ancestor of a matching file)

```
typedef struct TreeNode {
    char *name;
    int depth;
    int is_dir;
    struct TreeNode *next;
} TreeNode;
```

Algorithm Flow

1. **Argument Parsing:** `getopt()` processes `-w`, `-f`, `-b`, `-t`, `-p`, `-l` options. The program validates that `-w` is provided and at least one filter is active.
2. **Signal Handling:** A `SIGINT` handler is installed via `sigaction()`. It sets a `volatile sig_atomic_t` flag. The main loop checks this flag at each iteration and exits gracefully when interrupted.
3. **Recursive Directory Traversal:** The `search_recursive()` function:
   - Opens the directory with `opendir()`
   - Adds the directory as a hidden node (`is_dir=1`) to the linked list
   - Iterates entries with `readdir()`, skipping `.` and `..`
   - Calls `lstat()` (not `stat()`) on each entry to avoid following symbolic links
   - For directories: recurses first, then checks if the directory itself matches
   - For files: checks criteria and adds matching files to the list
   - Always traverses the entire subtree regardless of matches
4. **Ancestor Marking:** When a file matches, `mark_ancestors()` walks backward through the linked list to mark all ancestor directories as `is_dir=2`, ensuring they appear in the output tree.
5. **Tree Output:** `print_results()` iterates the linked list and prints only nodes with `is_dir==2` (ancestor directories) and `is_dir==0` (matching files). The indentation formula is: `|` followed by `(depth * 4 - 2)` dashes, then the entry name.
6. **Cleanup:** `free_tree_list()` frees all nodes and their `strdup`'ed names. This is called on all exit paths (normal, no match, SIGINT).

Design Decisions

Custom Regex Implementation ('+' Operator)

The assignment explicitly prohibits using `regcomp()`/`regexec()`. I implemented `match_pattern()` as a linear two-pointer algorithm:

- **Pattern pointer (`pi`)** and **string pointer (`si`)** advance through their respective strings
- When `pattern[pi+1] == '+'`: the character at `pattern[pi]` must appear one or more times. The algorithm consumes all consecutive matching characters, then advances `pi` by 2

- When no `+` follows: a simple character-by-character comparison is performed
- Both pointers must reach the end simultaneously for a full match
- All comparisons use `tolower()` for case-insensitive matching

This approach handles multiple `+` operators (e.g., `los+t+file`) correctly and runs in $O(n)$ time.

lstat() vs stat()

`lstat()` is used instead of `stat()` throughout the program. This is essential because:

- `stat()` follows symbolic links and returns information about the target file
- `lstat()` returns information about the link itself
- This allows the `-t l` filter to correctly identify symbolic links

Linked List for Tree Output

Instead of printing results immediately during traversal, all entries are collected in a linked list first. This two-pass approach was chosen because:

- The tree output requires knowing which directories are ancestors of matching files
- A directory's relevance can only be determined after its entire subtree has been searched
- This avoids printing directory paths that contain no matches

Permission Parsing

The `parse_permissions()` function converts a 9-character permission string (e.g., `rwxr-xr--`) into a POSIX `mode_t` bitmask by mapping each position to the corresponding `S_IR*`, `S_IW*`, `S_IX*` constants. The comparison uses `st_mode & 0777` to isolate the standard permission bits (excluding setuid/setgid/sticky bits).

Signal Safety

The `SIGINT` handler only sets a `volatile sig_atomic_t` flag — it does not call `printf()`, `free()`, or any other non-async-signal-safe function. The main program loop checks this flag and performs cleanup in normal execution context.

Memory Management

Every `malloc()` and `strdup()` call has a corresponding `free()` path:

- `add_tree_node()` allocates both the node and a copy of the name
- `free_tree_list()` frees both in reverse
- On allocation failure, partial allocations are immediately freed
- `free_tree_list()` is called on all exit paths (normal, no match, SIGINT)

System Call Error Handling

All system calls are checked for failure and the user is notified via `stderr`:

- `malloc()` / `strdup()` — checked for `NULL`, reported with `perror()`
- `sigaction()` — checked for `-1` return, reported with `perror()`
- `opendir()` — checked for `NULL`, reported with `fprintf(stderr,...)` and `perror()`
- `lstat()` — checked for `-1` return, reported with `fprintf(stderr,...)` and `perror()`
- `snprintf()` — checked for path truncation, reported with `fprintf(stderr,...)`
- `closedir()` — checked for `-1` return, reported with `fprintf(stderr,...)` and `perror()`

Requirements Achieved

#	Requirement	Status
1	Recursive traversal of entire subtree	Achieved
2	`-f` filename filter with case-insensitive custom `+` regex	Achieved
3	`-b` file size filter (exact byte match)	Achieved
4	`-t` file type filter (d/f/l/b/c/p/s)	Achieved
5	`-p` permissions filter (9-character string)	Achieved
6	`-l` hard link count filter	Achieved
7	`-w` mandatory search path	Achieved
8	Multiple `+` operators in pattern (e.g., `los+t+file`)	Achieved
9	`getopt()` for argument parsing	Achieved
10	Tree-formatted output matching the example	Achieved
11	"No file found" message when no matches	Achieved
12	Error messages printed to stderr	Achieved
13	All system calls checked for failure	Achieved
14	Usage information on missing/invalid arguments	Achieved
15	CTRL-C (SIGINT) graceful exit with resource cleanup	Achieved
16	No memory leaks (verified with valgrind)	Achieved
17	No zombie processes (single-process, no fork)	Achieved
18	No deadlocks (no synchronization primitives used)	Achieved
19	Compilation with `-Wall` and zero warnings	Achieved
20	Makefile compiles only, does not run	Achieved
21	`lstat()` used to avoid following symbolic links	Achieved

Requirements Not Achieved

All requirements specified in the homework assignment have been successfully implemented. No known failures or missing features exist at the time of submission.

Compilation and Testing

Compilation

```
make
# Equivalent to: gcc -Wall -o myFind myFind.c
```

Test Examples

```
# Search for files matching 'lost+file' pattern
./myFind -w /path/to/dir -f 'lost+file'

# Search for regular files of exactly 100 bytes
./myFind -w /path/to/dir -t f -b 100

# Search for directories
./myFind -w /path/to/dir -t d

# Search for files with specific permissions
./myFind -w /path/to/dir -p rwxr-xr--

# Search for files with 3 hard links
./myFind -w /path/to/dir -l 3

# Search for symbolic links
./myFind -w /path/to/dir -t l

# Search for sockets
./myFind -w /path/to/dir -t s

# Search for named pipes (FIFOs)
./myFind -w /path/to/dir -t p

# Combined criteria
./myFind -w /path/to/dir -f 'config+' -t f -b 0

# Memory leak check
valgrind --leak-check=full ./myFind -w /path/to/dir -t f
```

File Structure

```
myFind.c    -- Single source file containing all program logic
Makefile    -- Compilation rules only (does not run the program)
report.tex  -- LaTeX source of this report
report.pdf  -- This design report (compiled from report.tex)
```